Platform engineering

Build the internal developer platform: golden paths, a developer portal and catalog, self-service workflows, platform APIs, guardrails, multi-tenancy, cost allocation, and the measures that show it helps.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What platform engineering is, and how it differs from DevOps and SRE (12 min)
2. Platform as a product: users, roadmap and feedback (13 min)
3. Golden paths and paved roads (13 min)
4. The anatomy of an internal developer platform (14 min)
5. Developer portals and the service catalog (14 min)
6. Scaffolding, templates and self-service workflows (14 min)
7. Platform APIs with Crossplane and operators (14 min)
8. Guardrails, policy as code and secure defaults (13 min)
9. Environments, ephemeral previews and multi-tenancy (14 min)
10. Cost allocation, showback and unit economics (13 min)
11. Measuring developer experience: DORA, SPACE and cognitive load (14 min)
12. Platform SLOs, support and on-call for the platform team (13 min)
13. Adoption, migration and deprecation (13 min)
14. Capstone: design and ship a golden path (20 min)
Cheat sheet

Lesson 1. What platform engineering is, and how it differs from DevOps and SRE

12 minutes. Platform engineering builds the shared tools and paved paths that other engineering teams use to ship and run software, treating those tools as a product with customers rather than a ticket queue.

Every product team needs the same things to ship software: a repository, a pipeline, somewhere to run, secrets, logs, alerts and a way to ask for a database. If each team builds these alone, the company pays for the same work many times and gets many slightly different, slightly broken results. Platform engineering is the answer to that waste: one team builds the common parts once, and makes them easy to use.

What you will learn

- What a platform is and what it is not
- How platform engineering relates to DevOps and SRE
- The problems it solves and the ones it creates
- The signs that a company is ready for a platform team

A platform, in plain words

The CNCF platforms white paper describes a platform as an integrated collection of capabilities, presented according to the needs of its users. The important words are "integrated" and "users". A pile of tools is not a platform. A platform is a set of tools, services and documentation that fit together, with a clear front door, built for internal customers: the developers who build and run the company's products.

A platform is not a rebrand of the operations team, a single product such as Kubernetes, or a mandate that everyone must use one stack. If developers must file tickets and wait, you have a service desk. If they can get what they need in minutes without knowing how it works underneath, you have a platform.

How it relates to DevOps and SRE

DevOps is a culture and a set of practices: shared ownership of delivery and operation, automation, fast feedback. SRE applies software engineering to reliability, with service level objectives, error budgets and a focus on toil. Platform engineering is a way of organising the work so that these practices are easy to follow at scale.

	Main question	Typical output
DevOps	How do we build and run software together?	Shared practices, pipelines, ownership
SRE	How reliable should this service be, and how do we keep it so?	SLOs, error budgets, incident process
Platform engineering	How do we make the right way the easy way for every team?	Self-service tools, golden paths, a portal

The three overlap. A platform team often includes people with SRE skills, and a good platform bakes in SLO dashboards and safe delivery. The difference is the customer: an SRE team is usually attached to services, while a platform team serves other engineers.

The problem it solves: cognitive load

When "you build it, you run it" meets a modern cloud stack, a product developer is asked to know Kubernetes, Terraform, IAM, networking, observability, security scanning and the pipeline, on top of their own domain. The mental effort is called cognitive load. Teams slow down, copy-paste configuration they do not understand and make unsafe choices. A platform removes the undifferentiated parts of that load so developers can spend attention on the product.

MARGA by EknathaLabs
marga.eknathalabs.com

The problems it creates

Platforms have failure modes of their own. A team that builds what it finds interesting, not what users need, ships a platform nobody adopts. A platform that is mandatory but poor becomes a bottleneck. A platform that hides everything leaves developers helpless during an incident. Later lessons deal with each: treating the platform as a product, golden paths that can be left, and measurable adoption.

Signs you are ready

A platform team makes sense when several product teams repeat the same setup work, new engineers need weeks to deploy for the first time, security and compliance checks differ from team to team, or the operations team spends most of its time on repetitive requests. In a company with two teams and one service, a shared repository of templates is enough, and a dedicated platform team would be a cost without a customer.

A small thinking tool

Before building anything, write down the customer, the job and the current cost. The snippet below turns a few observations into a rough yearly estimate of wasted effort, which is the kind of number a platform proposal needs.

```
teams = 12
setups_per_team_per_year = 3
hours_per_setup = 16
hourly_cost = 70

hours = teams * setups_per_team_per_year * hours_per_setup
print("hours repeated each year:", hours)
print("rough cost:", hours * hourly_cost)
```

The numbers are invented; use your own. The point is to size the repeated work before proposing a team to remove it.

Diagram: Where a platform sits

Part	What it does	What goes wrong
Product teams	Build and run their own services.	Overloaded with infrastructure detail.
Front door	Portal, CLI and templates.	Unclear or hidden entry points.
Platform capabilities	Delivery, runtime, data, secrets and observability.	A pile of tools that do not fit together.
Cloud and clusters	The infrastructure underneath.	Details leaking up to users.
Platform team	Builds, supports and measures the platform.	Behaves like a ticket queue.

Flow: Product teams to Front door / Front door to Platform capabilities / Platform capabilities to Cloud and clusters / Platform team to Platform capabilities / Product teams to Platform team

Incident: The platform nobody used

> **Incident:** *A company built an internal platform over a year. Six months after launch, only two of fourteen teams used it, and the rest kept their own pipelines.*

```
$ platformctl adoption --by team
team          uses-platform   last-deploy-via
payments      no              own-pipeline
search        no              own-pipeline
checkout      yes             platform
identity      no              own-pipeline
(this is a simulated illustration, not a real incident)
```

The team had designed the platform around its own preferences and had never interviewed the product teams. Most teams found the first deploy slower than their current pipeline. The platform team then spent a month with three product teams, found that the slowest step was getting a database, and rebuilt the front door around that. Adoption rose because the platform now removed a pain people already had.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Renaming the operations team and changing nothing else
- Building tools because they are interesting, not because users need them
- Making the platform mandatory before it is good
- Hiding so much that developers cannot debug at 3am
- Starting a platform team when there is no repeated work to remove

Check yourself

What is the main customer of a platform team?

Answer: Internal engineering teams that build and run products

Which description best separates platform engineering from SRE?

Answer: Platform engineering builds self-service capabilities for other teams, while SRE focuses on service reliability

What problem does a platform mainly reduce?

Answer: Cognitive load on product developers

Which sign suggests a company is ready for a platform team?

Answer: Many teams repeat the same setup work and onboarding takes weeks

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Platform as a product: users, roadmap and feedback

13 minutes. Run the platform like a product: know your users, find their biggest pain, ship the thinnest useful version, publish a roadmap and measure whether people choose it when they could do otherwise.

The most common reason a platform fails is not technical. It is that nobody treated it as something people have to want. Internal users usually have a choice, even when it is unofficial: they can keep their own scripts, buy a managed service or ignore the platform and ask a friendly engineer for help. A platform that wins has to be better than those alternatives.

What you will learn

- How to identify users and their jobs
- How to find the pain worth fixing first
- The idea of the thinnest viable platform
- How to run a roadmap, feedback and release notes

Start with users, not tools

List the groups who use or are affected by the platform: product developers, data engineers, security reviewers, new joiners, on-call engineers. For each, write the jobs they are trying to get done: "deploy a change to production", "get a database for a prototype", "see why my service is slow". Interview a handful of them and watch them work. You will find friction that no survey would reveal, such as a copied command that everyone has to edit by hand.

A useful habit is to record each interview as a short journey: step, time taken, who helped, what went wrong. Add the steps up. The slowest and most painful steps across many teams are candidates for the first release.

Rank by pain and reach

For each problem, estimate how many teams have it, how often, and how much time it costs. Multiply roughly and sort. This avoids two common errors: fixing the loudest complaint, and fixing the most interesting problem. The ranking also gives you a defensible answer when someone asks why their request is not first.

```
problems = [
    ("get a database", 11, 6, 4.0),
    ("add a pipeline", 12, 3, 6.0),
    ("rotate a secret", 8, 12, 0.5),
    ("create a dashboard", 9, 2, 2.0),
]
# teams affected, times per year, hours each time
ranked = sorted(problems, key=lambda p: p[1] * p[2] * p[3], reverse=True)
for name, teams, times, hours in ranked:
    print(f"{name:20} {teams * times * hours:6.0f} hours per year")
```

The thinnest viable platform

Team Topologies uses the phrase "thinnest viable platform" for the smallest set of capabilities that lets product teams move faster. It might be a documented wiki page at first, then a template, then a self-service workflow. Each step earns the next. Avoid the temptation to build the full architecture diagram before anyone has used a single piece.

Make adoption a choice

Mandates give you usage but hide quality. If you can, let teams opt in, and watch what they choose. A rising share of new services created through the platform, and teams coming back for the second and third thing, are better signals than a mandate. Where a mandate is justified, for example a security control, make it narrow and explain why.

MARGA by EknathaLabs
marga.eknathalabs.com

A roadmap people can read

Publish a short roadmap with three horizons: now, next and later. State the problem each item solves and the users it helps, not only the technology. Say what you are not doing and why. Keep a visible changelog with release notes written for users: what changed, what they need to do and what they gain.

Feedback loops

Collect feedback in more than one way: an office hour, a channel with an on-call rotation for questions, a short survey after common tasks, and direct interviews each quarter. Close the loop. When someone reports a problem, tell them what happened. A platform team that never answers will soon stop being told.

Name an owner and a customer for each capability

Each capability needs a product owner on the platform side and, ideally, a named pilot customer on the product side. The pilot customer tells you whether the capability solves a real job, and becomes a reference for others.

Diagram: The platform product loop

Part	What it does	What goes wrong
Users and jobs	Interviews and observation of real work.	Assumptions instead of watching users.
Ranked pain	Reach times frequency times cost.	The loudest request wins by default.
Thinnest slice	The smallest capability that removes the pain.	A big design before first use.
Release and notes	Docs, a changelog and an announcement.	Users never hear about it.
Adoption and feedback	Usage, satisfaction and time saved.	Counting mandated use as success.

Flow: Users and jobs to Ranked pain / Ranked pain to Thinnest slice / Thinnest slice to Release and notes / Release and notes to Adoption and feedback / Adoption and feedback to Users and jobs

Incident: The request queue that ate the roadmap

> **Incident:** *A platform team answered every request in order, and after a quarter had shipped nothing that reduced requests.*

```
$ platformctl requests --last 90d --by type
database-request      214
pipeline-setup        167
access-request         98
namespace-request      76
(this is a simulated illustration, not a real incident)
```

The team had treated itself as a support desk. Looking at the counts, they saw that two request types made up most of the work and both were repeatable. They paused new features for a month, automated the database and pipeline requests as self-service workflows, and watched the queue fall by more than half the next quarter. The measure of success became requests that no longer needed to exist.

Common mistakes

- Prioritising by who asks loudest
- Building the whole platform before anyone uses a piece of it
- Counting mandated usage as proof of value
- Publishing no roadmap or release notes
- Never closing the loop with people who gave feedback

Check yourself

Why interview and watch users before building?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: To find real friction that surveys and assumptions miss

What does the thinnest viable platform mean?

Answer: The smallest set of capabilities that makes product teams faster

How should you rank candidate improvements?

Answer: By how many teams are affected, how often and how much time it costs

Which signal best shows real adoption?

Answer: Teams choosing the platform for new work without being forced

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Golden paths and paved roads

13 minutes. A golden path is the supported, well-lit way to build and run a common kind of service. Learn how to choose, design and maintain paths that people take because they are easier, not because they are forced.

Every team wants to move fast and nobody wants to rediscover how to set up logging. A golden path, sometimes called a paved road, is the platform's answer: a documented, automated, opinionated way to go from an idea to a running, observable, secure service. It is a path, not a wall. People can step off it when they have a good reason, and they know what they give up when they do.

What you will learn

- What a golden path contains
- How to choose which paths to build
- How to design for the common case and still allow exits
- How to keep a path alive as technology changes

What a golden path is made of

A good path covers the whole journey for one kind of workload, for example "a stateless HTTP service in Go" or "a scheduled data job in Python". Typically it includes:

- A repository template with the right layout and tooling
- A pipeline with build, test, scan and deploy steps
- A runtime definition with sensible resources, probes and scaling
- Default logging, metrics, tracing and an initial dashboard
- Security basics such as an image policy and secret handling
- Documentation and an owner field in the catalog

Each piece is something the team would otherwise assemble by hand. Together they mean the first deploy takes an afternoon, not weeks.

Choose paths by demand

Look at what teams build. If 60 percent of new services are stateless web APIs, build that path first. Do not build a path for a one-off workload. A short list of paths that cover most cases beats a long list that is hard to maintain. A reasonable starting set is: web service, background worker, scheduled job and static or frontend site.

Opinionated, with exits

A path works because it makes decisions for the user: which language versions, which base image, which pipeline stages, which dashboards. Too many options recreate the problem. But a path with no exit forces teams with unusual needs to fight it or leave. The usual pattern is to make the path the default and supported option, and offer a documented way to customise individual layers, for example adding an extra sidecar or a different build step, while keeping the guardrails that matter: ownership metadata, scanning and observability.

State what support looks like on and off the path. On the path, the platform team handles upgrades and answers questions. Off the path, the team owns more, and the platform may only support the common pieces.

Make the right thing the easy thing

Do not rely on rules and review to push people onto the path. Rely on convenience. If the path creates a repository, wires the pipeline and shows a green deploy in ten minutes, people will use it. If it needs three tickets, they will not.

MARGA by EknathaLabs marga.eknathalabs.com

Check what a service has

You can measure how much of a service follows the path by checking its metadata. The snippet below counts which required pieces a service declares.

```
required = {"owner", "pipeline", "dashboard", "runbook", "slo"}

services = {
    "checkout": {"owner", "pipeline", "dashboard", "runbook", "slo"},
    "search": {"owner", "pipeline", "dashboard"},
    "legacy-report": {"pipeline"},
}

for name, have in services.items():
    missing = sorted(required - have)
    status = "on path" if not missing else "missing " + ", ".join(missing)
    print(f"{name:14} {status}")
```

Keep the path alive

A path is a product with a lifecycle. Version it. When a language version, base image or library has a security fix, update the template and give existing services an automatic pull request, so they stay current without manual work. Deprecate old versions with a date and a migration guide. Track how many services use each version, because a path that nobody updates becomes a trap.

Avoid the two failure modes

The first is the abandoned path: built with fanfare, never updated, now a source of outdated defaults. The second is the cage: a path so rigid that every real need requires an exception. Review exceptions regularly. A repeated exception is a signal to improve the path.

Diagram: A golden path for a web service

Part	What it does	What goes wrong
Template	Repository with layout, tests and tooling.	Outdated language or base image.
Pipeline	Build, test, scan and deploy.	Steps copied and edited by hand.
Runtime	Defaults for resources, probes and scaling.	Defaults that do not fit the workload.
Observability	Logs, metrics, traces and a dashboard.	A dashboard nobody looks at.
Security	Image policy and secret handling.	Checks that can be skipped silently.
Catalog entry	Owner, tier and runbook link.	Metadata that is never filled in.

Flow: Template to Pipeline / Pipeline to Runtime / Runtime to Observability / Runtime to Security / Observability to Catalog entry / Security to Catalog entry

Incident: The cage and the escape

> **Incident:** *A golden path allowed only one language version. A team needing a newer runtime for a security fix could not get it, so it built its own pipeline.*

```
$ platformctl path status web-service
path version:    3.2
runtime pinned:  17
services on 3.2: 41
services off path: 9 (7 cite "runtime version")
(this is a simulated illustration, not a real incident)
```

The platform team saw that nine services had left the path and seven gave the same reason. They added a supported choice of two runtime versions, with an automatic upgrade pull request for the older one. Within two months most of the nine had come back, because staying on the path was now easier than maintaining their own pipeline.

Common mistakes

- Building a path for a workload only one team has

MARGA by EknathaLabs marga.eknathalabs.com

- Giving the path no exit, so teams leave it entirely
- Forgetting to update templates after launch
- Counting services that used the template once as "on the path" years later
- Measuring paths by how many rules they enforce instead of how fast they get people to production

Check yourself

What is a golden path?

Answer: A supported, automated way to build and run a common kind of workload

How should you choose which paths to build first?

Answer: By the kinds of workloads most teams build

What keeps a path from becoming a trap over time?

Answer: Versioning, automatic update pull requests and dated deprecations

Why offer a documented way to customise a path?

Answer: So teams with unusual needs stay on supported ground instead of leaving

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. The anatomy of an internal developer platform

14 minutes. See the layers of an internal developer platform: developer control, delivery, resource orchestration, monitoring and security, and how they connect so that one request creates a working, governed service.

An internal developer platform, or IDP, is not one product. It is a set of layers that together let a developer ask for something and get a safe, working result. Understanding the layers helps you decide what to build, what to buy and where the seams will hurt.

What you will learn

- The main planes of a platform and what each does
- How a single request flows through them
- Where open source tools commonly fit
- How to avoid rebuilding everything at once

The planes

Reference architectures from the CNCF and from vendors describe slightly different layers, but the idea is stable. A useful five-part view is:

Plane	Purpose	Examples of what lives here
Developer control	Where people ask for things	Portal, command line, templates, catalog
Integration and delivery	Turns code into running software	Source control, pipelines, registries, deployment tools
Resource orchestration	Creates the infrastructure behind requests	Infrastructure as code, Crossplane, operators, cluster provisioning
Monitoring and logging	Shows what is happening	Metrics, logs, traces, dashboards, alerts
Security	Controls identity, secrets and policy	Identity provider, secret store, policy engine, scanners

The labels vary, but a request always touches all of them: a developer asks for a service, a pipeline builds it, infrastructure appears, monitoring is wired up and security checks run.

Follow one request

Suppose a developer wants a new web service with a database.

1. They pick a template in the portal and fill in a name and owner.
2. The platform creates a repository and registers the service in the catalog.
3. The pipeline builds the first version, scans it and deploys to a development environment.
4. A resource request creates a database through the orchestration layer, with credentials stored in the secret store.
5. Default dashboards and alerts appear because the template declared them.
6. Policy checks confirm the service has an owner, resource limits and an approved image.

If any step needs a ticket, the platform has a gap. Mapping a request like this is a good way to find the gaps.

MARGA by EknathaLabs marga.eknathalabs.com

A catalog at the centre

Most successful platforms keep a catalog: a record of every service, its owner, its dependencies and its tier. The catalog is what other planes read. The pipeline reads the owner to route failures. Monitoring reads the tier to set alert severity. Policy reads the data class to decide what is allowed. Without it, each tool keeps its own incomplete picture.

Compose, do not monolith

You do not need to build a platform in one go, and you rarely need to build every layer yourself. Most platforms are composed of existing tools with a thin layer of glue and a good front door. Decide, for each capability, whether to build, buy or adopt open source, and write down why. The seams between tools are where reliability and user experience break, so spend your effort there.

Standards at the seams

Use open standards where they exist so that parts can be replaced: container images and registries follow OCI specifications, telemetry can use OpenTelemetry, deployments can be declared as Kubernetes resources, and identity can use OpenID Connect. Each standard lowers the cost of changing one part later.

Model the platform's contract

Describe each capability as a contract: what the user asks for, what they get, what the service level is and who to contact. The snippet below checks that a list of capabilities has the fields a contract needs.

```
capabilities = [
    {"name": "new-service", "input": "template, name, owner", "output": "repo, pipeline, dev env", \
     "slo": "ready in 15 minutes", "owner": "platform-delivery"},
    {"name": "database", "input": "engine, size, env", "output": "instance and secret", "slo": "re \
    ady in 10 minutes", "owner": "platform-data"},
    {"name": "dashboard", "input": "service", "output": "default dashboard"},
]

need = {"name", "input", "output", "slo", "owner"}
for c in capabilities:
    gap = sorted(need - set(c))
    print(c["name"], "complete" if not gap else "missing " + ", ".join(gap))
```

Diagram: One request across the planes

Part	What it does	What goes wrong
Developer control	Portal, command line and templates.	A form that does not match what the system accepts.
Catalog	Owners, tiers and dependencies.	Stale or incomplete records.
Delivery	Pipeline, registry and deployment.	Skipped checks.
Resource orchestration	Infrastructure created on request.	Resources with no owner.
Monitoring	Dashboards and alerts from defaults.	Alerts with no routing.
Security and policy	Identity, secrets and rules.	Rules applied in one plane only.

Flow: Developer control to Catalog / Developer control to Delivery / Delivery to Resource orchestration / Resource orchestration to Monitoring / Catalog to Security and policy / Security and policy to Delivery

Incident: The ticket hidden in the flow

> **Incident:** *A new platform advertised "service in 15 minutes", yet teams reported waiting days. Nobody could say where the time went.*

```
$ platformctl trace new-service req-5530
portal-submit       00:00
repo-created        00:02
pipeline-first-run  00:09
database-requested  00:12
database-approved   2d 04:10   (manual approval)
```

MARGA by EknathaLabs marga.eknathalabs.com

```
(this is a simulated illustration, not a real incident)
```

Tracing one request showed that the database step waited for a manual approval by a person in another team. The platform team agreed a size limit and an automatic approval for small development databases, kept human review for production, and the end-to-end time for the common case dropped to under twenty minutes.

Common mistakes

- Building every plane before any user has tried the first one
- Keeping no catalog, so each tool has its own partial picture
- Ignoring hand-offs and manual approvals in the middle of a flow
- Locking into a tool with no standard interface
- Describing capabilities by technology instead of by what users get

Check yourself

What does the developer control plane provide?

Answer: The front door where people request and see things

Why keep a catalog at the centre of a platform?

Answer: Other planes read it for owners, tiers and dependencies

How can you find gaps in a platform?

Answer: Trace one request end to end and look for manual hand-offs

Why prefer open standards at the seams between tools?

Answer: They make each part easier to replace later

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Developer portals and the service catalog

14 minutes. A developer portal is the front door to the platform and the service catalog is its memory. Learn what goes in the catalog, how to keep it accurate, and how tools such as Backstage use it.

Ask a large engineering organisation "who owns this service?" and the honest answer is often "let me ask around". A service catalog fixes that by keeping a living record of services, their owners and their links. A developer portal puts the catalog, the templates, the docs and the status of your services in one place. Together they are the visible part of a platform.

What you will learn

- What a catalog holds and why accuracy matters more than size
- How a portal such as Backstage is organised
- How catalog entries are defined and discovered
- How to keep entries from going stale

What belongs in a catalog

The minimum useful entry answers four questions about a service: who owns it, what is it, how do I run or reach it, and what does it depend on. Practical fields are:

- Name, description and lifecycle (experimental, production, deprecated)
- Owning team and an on-call contact
- Tier or criticality, and the data classes it handles
- Links to the repository, pipeline, dashboard, runbook and documentation
- Dependencies on other services, databases and queues

Other kinds of things can live in the same catalog: APIs, libraries, infrastructure resources, teams and people. The relationships between them are what make it more than a spreadsheet.

How Backstage models it

Backstage is an open source developer portal, started at Spotify and now a CNCF project. Its software catalog is built from descriptor files, typically named `catalog-info.yaml`, stored next to the code. Each describes an entity with a kind, such as Component, API, Resource, System, Domain, Group or User.

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: checkout
  description: Handles cart and payment orchestration
  annotations:
    github.com/project-slug: example-org/checkout
spec:
  type: service
  lifecycle: production
  owner: team-payments
  system: shop
  dependsOn:
    - resource:default/orders-db
```

Because the descriptor lives with the code, the owner of the service updates it in the same pull request that changes the service. The portal discovers files from source control and keeps the catalog in sync. This is a more reliable approach than a central team editing a spreadsheet.

MARGA by EknathaLabs (marga.eknathalabs.com)

More than a catalog

A portal usually adds more surfaces on top of the catalog. Software templates create new services from a form. Documentation systems such as TechDocs render Markdown from the repository. Plugins show pipeline status, deployments, alerts, cost and security findings against each service. The value is in seeing everything about a service on one page without opening six tools.

Accuracy beats volume

A catalog with ten thousand entries of unknown quality is worse than one with a hundred accurate entries. Wrong owners send people to the wrong team at 3am. Design for accuracy:

- Make ownership required, and reject entries without a valid team
- Store descriptors with the code and review them
- Check links and owners automatically and report broken ones
- Mark entries not updated for a long time and ask the owner to confirm or retire them
- Remove or archive services that no longer exist

Run a freshness check

The snippet below flags catalog entries whose owner does not exist or that have not been confirmed recently.

```
teams = {"team-payments", "team-search", "team-platform"}
today = 400  # day number, for the example
entries = [
    {"name": "checkout", "owner": "team-payments", "confirmed": 390},
    {"name": "legacy-report", "owner": "team-old", "confirmed": 120},
    {"name": "search-api", "owner": "team-search", "confirmed": 250},
]

for e in entries:
    problems = []
    if e["owner"] not in teams:
        problems.append("owner not found")
    if today - e["confirmed"] > 180:
        problems.append("not confirmed in 180 days")
    print(f"{e['name']:14}", ", ".join(problems) or "ok")
```

Make it useful before it is complete

Start with the services people ask about most, such as the critical tier. Show something people want in the portal early, for example a single page that links a service to its dashboard and runbook. When the catalog saves someone time during an incident, the owners will keep it accurate themselves.

Do not build what you can adopt

Building a portal from scratch is a large, permanent commitment. Backstage is a framework, so adopting it still takes engineering work to deploy, extend and maintain. Compare it with commercial portals and with a simpler option, such as a generated documentation site backed by a catalog file. Choose the smallest tool that gives you the catalog and the front door.

Diagram: From code to portal

Part	What it does	What goes wrong
Service repository	Holds code and its catalog descriptor.	Descriptor missing or outdated.
Catalog discovery	Reads descriptors from source control.	Rate limits or stale sync.
Software catalog	Entities, owners and relations.	Entries with no valid owner.
Developer portal	Search, pages, templates and docs.	Too many plugins, slow pages.
Developers and on-call	Find owners, runbooks and status.	Data they cannot trust.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Service repository to Catalog discovery / Catalog discovery to Software catalog / Software catalog to Developer portal / Developer portal to Developers and on-call

Incident: The page that went to the wrong team

> **Incident:** *At 02:10 an alert on the reporting service paged a team that had not owned it for a year, and nobody realised for forty minutes.*

```
$ platformctl catalog show reporting-api
owner:      team-old   (team not found in directory)
lifecycle:  production
confirmed:  2025-02-03
(this is a simulated illustration, not a real incident)
```

The service had moved teams during a reorganisation, but the descriptor kept the old owner and nothing checked it against the team directory. After the incident the platform team added a nightly check that failed any entry whose owner is not a real team, and a monthly report of entries not confirmed in six months.

Common mistakes

- Treating a catalog as a one-off data import
- Letting a central team edit entries instead of the service owners
- Allowing entries with no owner
- Installing many portal plugins before the catalog is trustworthy
- Building a portal from scratch when an existing one would do

Check yourself

Why store catalog descriptors next to the code?

Answer: The owners update them in the same change that alters the service

What is the minimum a catalog entry should say?

Answer: Who owns it, what it is, how to reach or run it and what it depends on

Which is better, a large catalog of unknown quality or a smaller accurate one?

Answer: A smaller accurate one

How can you detect stale entries?

Answer: Automatic checks for missing owners and entries not confirmed for a long time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Scaffolding, templates and self-service workflows

14 minutes. Turn the golden path into something a developer can start in minutes: parameterised templates that create repositories, pipelines and catalog entries, with validation, approvals and an audit trail.

A path on paper is not much help. People follow the path that is one click away. Scaffolding is the mechanism that turns a request such as "new Go service called billing, owned by team payments" into a repository, a working pipeline, a catalog entry and a first deployment, without anyone filing a ticket.

What you will learn

- What a good template asks for and generates
- How to validate input and apply policy before creating anything
- How self-service differs from "self-serve with approval"
- How to keep generated repositories updatable

Ask few questions

A template form should ask only what it cannot infer: a name, an owning team, a short description, maybe the language and a tier. Fewer fields mean faster use and fewer mistakes. Derive everything else: repository name from service name, namespace from team, alert routing from owner. Offer safe defaults and hide advanced options behind an extra step.

Validate before you create

Check input early and give clear errors. Names should follow the naming convention and be unique. The team must exist. The tier must be one of the allowed values. A template that fails halfway, leaving a half-created repository, is worse than one that refuses at the start.

```
import re

TEAMS = {"payments", "search", "platform"}
TIERS = {"critical", "standard", "experimental"}

def validate(req):
    errors = []
    if not re.fullmatch(r"[a-z][a-z0-9-]{2,29}", req.get("name", "")):
        errors.append("name must be 3-30 lowercase letters, digits or hyphens, starting with a let \
    ter")
    if req.get("team") not in TEAMS:
        errors.append("team is not known")
    if req.get("tier") not in TIERS:
        errors.append("tier must be one of " + ", ".join(sorted(TIERS)))
    return errors

print(validate({"name": "billing", "team": "payments", "tier": "standard"}))
print(validate({"name": "Billing!", "team": "ghost", "tier": "gold"}))
```

What a template generates

A template can create several things in one flow: a repository from a skeleton, with the service name substituted; a pipeline definition; deployment manifests; a catalog descriptor; a documentation stub; monitoring dashboards and alert rules; and access for the owning team. Order the steps so that if one fails, earlier work can be undone or the flow can be resumed. Make each step idempotent, so running it twice does not create duplicates.

MARGA by EknathaLabs
marga.eknathalabs.com

Self-service and approval

Self-service does not mean no control. Divide requests by risk. Low-risk, reversible requests such as a development namespace or a small database should be instant. Higher-risk requests, such as production access or large resources, can require approval, but the approval should be a step in the workflow with context shown to the approver, not a manual hand-off by email. Record who asked, who approved and what was created.

Templates must stay updatable

Generating a repository once and walking away leaves hundreds of copies that drift. Plan for updates:

- Keep shared pipeline logic in a versioned, reusable component that repositories reference instead of copy
- Record which template version created each repository
- Offer an automatic pull request that applies template changes to existing repositories
- Test the template itself in CI by generating a sample and building it

Test the template

A template is code, so test it. Generate a service from the template in CI, build it, run its tests and deploy it to a scratch environment. A broken template breaks every new service, which makes it one of the most valuable things to test.

Keep the experience consistent

Whether a developer uses the portal form, a command line or an API, the same workflow should run. Do not build separate logic for each front end. Define the workflow once and give it several entrances.

An audit trail from the start

Every run records its inputs, the identity of the requester, the policy decisions and the resources created. When something goes wrong, you can answer "what created this, and who asked for it?" in a minute.

Diagram: A self-service workflow

Part	What it does	What goes wrong
Request form	Name, team, language and tier.	Too many fields.
Validation and policy	Input checks and rules before anything is created.	Checks that run after creation.
Approval if needed	Context-rich review for higher-risk requests.	Approval by email with no context.
Create resources	Repository, pipeline, catalog entry and access.	Steps that cannot be repeated safely.
Audit record	Inputs, approver and results.	Missing records.
Developer	Gets a working service quickly.	Waits on hidden manual steps.

Flow: Request form to Validation and policy / Validation and policy to Approval if needed / Validation and policy to Create resources / Approval if needed to Create resources / Create resources to Audit record / Create resources to Developer

Incident: Half a service

> **Incident:** *A template failed after creating the repository but before registering the owner. For two weeks a production-bound service existed with no catalog entry and no owner.*

```
$ platformctl workflow show run-7714
step 1 create-repo     ok
step 2 add-pipeline    ok
step 3 register-owner  error: team directory timeout
step 4 first-deploy    skipped
(this is a simulated illustration, not a real incident)
```

MARGA by EknathaLabs marga.eknathalabs.com

The workflow stopped on step three and nobody was told. The platform team made each step idempotent, added automatic retry, and made failures notify the requester and the platform channel. They also added a nightly check that found any repository without a catalog entry.

Common mistakes

- Asking developers for fields the platform can derive
- Validating after creating resources
- Treating approval as an email, not a workflow step with context
- Copying pipelines into every repository instead of referencing shared ones
- Never testing the template itself

Check yourself

Why should a template validate input before creating anything?

Answer: A failure halfway leaves partial resources that are hard to clean up

Which requests should be instant self-service?

Answer: Low-risk, reversible ones such as a small development environment

How do you stop generated repositories drifting from the template?

Answer: Reference shared versioned pipeline components and offer automatic update pull requests

Why should each workflow step be idempotent?

Answer: So a retry does not create duplicates or fail

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Platform APIs with Crossplane and operators

14 minutes. Use the Kubernetes control plane pattern to offer your own infrastructure APIs: a developer files a small resource, a controller reconciles it into cloud resources, and the platform keeps it correct over time.

A form or a pipeline creates things once. A control plane keeps them right. Kubernetes popularised the idea: you declare the state you want, and controllers continuously work to make reality match. Platform teams reuse that pattern to offer their own APIs, such as `Database` or `Environment`, and let controllers do the work behind them.

What you will learn

- Why a declarative API suits infrastructure
- How Crossplane compositions map a simple request to cloud resources
- When to write an operator or controller instead
- How to keep the abstraction honest

Declare, then reconcile

In an imperative script, you tell the system what steps to run. In a declarative API, you say what should exist, and a controller compares desired state with actual state and acts on the difference. It repeats forever, so drift is corrected and failures are retried. The same loop that restarts a crashed pod can recreate a deleted bucket or resize a database.

This gives platforms three useful properties: a single source of truth for what was asked, automatic repair, and an API that other tools can read, such as the catalog or a pipeline.

Crossplane in outline

Crossplane extends Kubernetes so that cloud resources are managed as Kubernetes objects. The core ideas are:

- **Providers** that know how to talk to a cloud or service and expose its resources as managed resources
- **Composite resource definitions** that define your own API, with a schema
- **Compositions** that map one of your API objects onto several managed resources and decide the defaults

A developer writes a short object. The platform team has already decided what it expands into.

```
apiVersion: platform.example.org/v1alpha1
kind: Database
metadata:
  name: orders
  namespace: team-payments
spec:
  engine: postgres
  size: small
  environment: dev
```

Behind that, a composition might create a database instance, a subnet group, a backup schedule and a secret holding the connection details, with the size mapped to concrete instance classes. Crossplane's APIs and recommended structure have changed between versions, so use the documentation for the version you run when you write real definitions.

MARGA by EknathaLabs
marga.eknathalabs.com

Why not give people the cloud API directly?

Cloud APIs are wide and powerful. A developer who wants "a small Postgres" would have to choose among dozens of options, any of which might violate policy, cost too much or lack backups. A platform API narrows the choices to the ones the company supports, bakes in encryption, backups, tags and network placement, and gives each request a clear owner and namespace. It also lets you change the implementation, for example moving from one instance type to another, without every team rewriting its configuration.

When to write an operator

Compositions cover the common case of mapping one object to several resources. When you need custom logic, such as running a step-by-step migration, calling an internal system or managing a stateful workload, a custom controller or operator is the tool. Frameworks such as Kubebuilder help. Prefer composition when it is enough, because a custom controller is code you must test, run and upgrade.

Keep the abstraction honest

Abstractions leak. Decide in advance what the user can see and change, and what they cannot. Show the status of the underlying resources in the object's status, with clear conditions such as `Ready` and a message when something fails. Document the escape hatch for needs the abstraction does not cover. Version the API and plan migrations, because teams will depend on it.

Check a request against supported sizes

Here is a small stand-in for the logic a composition applies: map a friendly size to concrete settings and refuse anything outside the supported set.

```
SIZES = {
    "small": {"cpu": 2, "memory_gb": 8, "storage_gb": 50},
    "medium": {"cpu": 4, "memory_gb": 16, "storage_gb": 200},
}
ENGINES = {"postgres", "mysql"}

def expand(spec):
    if spec["engine"] not in ENGINES:
        raise ValueError("engine not supported: " + spec["engine"])
    if spec["size"] not in SIZES:
        raise ValueError("size not supported: " + spec["size"])
    plan = dict(SIZES[spec["size"]])
    plan.update(engine=spec["engine"], backups=True, encrypted=True, tags={"env": spec["environmen \
    t"]})
    return plan

print(expand({"engine": "postgres", "size": "small", "environment": "dev"}))
```

Operate the control plane like a product

The controllers are production software. Monitor reconcile errors and queue depth. Alert on resources stuck not-ready. Back up the control plane state. Plan upgrades of the controllers and providers, and test them against a copy of real resources first. A broken provider upgrade can stall changes everywhere.

Diagram: A request becomes cloud resources

Part	What it does	What goes wrong
Database object	A short request in the team's namespace.	Free-form options that bypass policy.
Platform API	Schema, defaults and validation.	Breaking changes without versions.
Composition	Maps the request to managed resources.	Hidden defaults nobody documented.
Provider	Calls the cloud API.	Credentials with too much access.
Cloud resources	Instance, backup, subnets and secret.	Drift from manual changes.
Status	Conditions reported back to the user.	Errors hidden in controller logs.

Flow: Database object to Platform API / Platform API to Composition / Composition to Provider / Provider to Cloud resources / Cloud resources to Status / Status to Database object

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Ready, but not really

> **Incident:** *A team created a Database object, saw it as "created", and began deploying. The instance never came up because of a quota limit, and the application failed for an hour.*

```
$ kubectl get database orders -n team-payments
NAME     READY   SYNCED   AGE
orders   False   True     41m
(this is a simulated illustration, not a real incident)
```

The object had been accepted, but the readiness condition was false because the provider hit a quota error that only appeared in controller logs. The platform team surfaced the cloud error message in the object's status, added an alert for resources not ready after ten minutes, and wrote the quota check into the pre-request validation.

Common mistakes

- Exposing the full cloud API instead of a narrow, supported one
- Writing a custom controller when a composition would do
- Hiding underlying errors from users
- Changing an API without versioning and a migration path
- Giving the provider far more cloud permissions than the APIs need

Check yourself

What does a controller do in a declarative system?

Answer: It compares desired and actual state and acts on differences, repeatedly

What is the role of a Crossplane composition?

Answer: To map a simple platform object onto several managed cloud resources with defaults

When is a custom operator a better choice than a composition?

Answer: When you need custom logic such as ordered migrations or calls to internal systems

Why report underlying errors in the object's status?

Answer: So users can see why a request is not ready without reading controller logs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Guardrails, policy as code and secure defaults

13 minutes. Safety that depends on people remembering rules fails. Build guardrails into the platform with secure defaults, admission policies and checks that explain themselves, so the safe path is also the easy one.

A guardrail keeps people from driving off the road without slowing the journey. In a platform, guardrails are checks and defaults built into the paths and workflows: images must come from an approved registry, services must have an owner, containers must not run as root. When they work, developers rarely notice them. When they are badly designed, they are the thing developers resent most.

What you will learn

- Defaults, guardrails and gates, and when to use each
- How policy as code works with admission control
- How to write policies that explain themselves
- How to roll out a new policy without breaking teams

Three levels of control

1. **Defaults.** The safe option is already chosen. A template sets resource limits and a non-root user. Most people never change them, so the default does most of the work.
2. **Guardrails.** Automated checks that block or warn on dangerous choices, with a clear message. A policy rejects a deployment with no resource limits.
3. **Gates.** A human approval for rare, high-risk actions, such as widening access to production data.

Prefer defaults first, guardrails next and gates last. Each step adds friction, so use the lightest control that gives enough safety.

Policy as code

Policies written in a document are forgotten. Policies written as code can be reviewed, tested, versioned and enforced automatically. In Kubernetes, admission controllers intercept requests to the API server and can validate or mutate them. Tools commonly used for this include Kyverno, Open Policy Agent with Gatekeeper, and the built-in validating admission policies, which use the Common Expression Language. The same rules can also run earlier, in pipelines and in the template step, so developers see failures in their own change instead of at deploy time.

An example rule, written in plain terms: every Deployment must declare resource requests and limits for each container. In a policy engine this becomes a few lines of configuration. The important part is where it runs and what it says when it fails.

Messages that help

A policy that rejects with "denied" teaches nothing. A good message says what was wrong, why the rule exists and how to fix it: "Container `web` has no memory limit. Limits stop one service from starving others. Add `resources.limits.memory`, or use the default from the path template." Link to the documentation. Developers read the message at the moment they are most motivated to fix the problem.

```
def check_deployment(d):
    problems = []
    for c in d["containers"]:
        if "limits" not in c.get("resources", {}):
            problems.append(f"container {c['name']} has no resource limits: add resources.limits s \
    o one service cannot starve others")
        if c.get("run_as_root", True):
            problems.append(f"container {c['name']} may run as root: set runAsNonRoot to true")
    if "owner" not in d.get("labels", {}):
        problems.append("deployment has no owner label: add the owning team so alerts can be route \
    d")
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    return problems

deploy = {
    "labels": {},
    "containers": [{"name": "web", "resources": {}, "run_as_root": True}],
}
for p in check_deployment(deploy):
    print("DENY:", p)
```

Roll out in stages

Switching a new policy to enforce on day one will break teams and cost you trust. Use stages:

1. Audit mode: record violations but allow everything
2. Publish the list of affected services and the fix
3. Warn mode: show the message but allow
4. Enforce, with a dated deadline and a documented exception process

Measure how many violations remain at each stage. If many teams hit the same violation, improve the template so the default prevents it.

Exceptions are part of the design

Real systems have exceptions. Make them explicit, time-limited and visible: an exception has an owner, a reason and an expiry date, and is listed somewhere reviewable. Silent, permanent exceptions erode the policy until it means nothing.

Test the policies

Write test cases for each policy, both a passing and a failing example, and run them in CI whenever the policy changes. A policy bug that rejects all deployments is an outage you caused yourself.

Do not duplicate trust

Run the same policy in more than one place only if it is the same code. Different copies drift. Keep one policy repository and let the pipeline, admission controller and template reference it.

Diagram: Guardrails along the path

Part	What it does	What goes wrong
Template defaults	Safe values chosen up front.	Defaults that are rarely updated.
Pipeline checks	Policy and scan results in the pull request.	Checks that can be bypassed.
Admission control	Validates requests to the cluster.	A rule that blocks everything.
Audit and exceptions	Violations and time-limited exceptions.	Permanent silent exceptions.
Developer	Sees clear messages and fixes the cause.	Messages that only say denied.

Flow: Template defaults to Pipeline checks / Pipeline checks to Admission control / Admission control to Audit and exceptions / Pipeline checks to Developer / Admission control to Developer

Incident: The rule that blocked a rollback

> **Incident:** *During an outage the on-call engineer tried to roll back a deployment, and an admission policy rejected the old version because it lacked a label added after it was built.*

```
$ kubectl rollout undo deployment/checkout -n shop
Error from server: admission webhook "policy.platform" denied the request: missing label owner
(this is a simulated illustration, not a real incident)
```

The new policy applied to old workloads as well as new ones. The incident lasted fifteen minutes longer while a platform engineer granted an exception. Afterwards the team made new policies apply only to resources created or changed after a date, added an emergency path with logging for rollbacks, and tested policies against old versions before enforcing them.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Relying on documents and reviews instead of automated checks
- Denying with a message that gives no fix
- Enforcing a new policy on day one
- Letting exceptions live forever without owners
- Running different copies of the same policy in different places

Check yourself

Which control should you prefer first?

Answer: A secure default that most users never change

What makes a policy failure message useful?

Answer: It says what is wrong, why the rule exists and how to fix it

How should you roll out a new enforced policy?

Answer: Audit, then warn, then enforce with a deadline and an exception process

Why test policy code?

Answer: A buggy policy can block valid changes, including emergency rollbacks

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Environments, ephemeral previews and multi-tenancy

14 minutes. Give teams isolated places to build and test without a cluster each: namespaces and quotas, per-change preview environments, clear promotion rules and cleanup that actually runs.

Teams need somewhere to run their code before production. Too few environments and people step on each other. Too many and the cloud bill and the maintenance work balloon. A platform decides how environments are shared, how isolated they are and how they are created and removed.

What you will learn

- Choices for sharing clusters between teams
- How namespaces, quotas and policies create soft isolation
- Ephemeral preview environments for each change
- Promotion paths and cleanup

Shared or separate

The main choices, from cheapest to most isolated, are:

Model	Isolation	Cost and effort
Namespace per team on a shared cluster	Soft, enforced by policy	Lowest
Virtual clusters per team	Separate API server, shared nodes	Medium
Cluster per team or per environment	Hard, separate control planes	Highest

Most platforms start with namespaces and move teams to stronger isolation when risk or scale demands it, for example for regulated workloads. The Kubernetes documentation on multi-tenancy describes the control plane and data plane concerns for each model.

Making namespaces behave like tenants

A namespace by itself is only a naming boundary. To make it a tenant, add:

- **RBAC** so a team can manage only its own namespaces
- **Resource quotas and limit ranges** so one team cannot consume the whole cluster
- **Network policies** with default deny between namespaces, and explicit allows
- **Pod security standards** to restrict dangerous pod settings
- **Default labels** for owner and cost allocation

The platform should create all of this when a team onboards, from one definition, so every tenant starts identical.

Ephemeral preview environments

A preview environment is a short-lived copy of the application created for a pull request, reachable on its own URL, and deleted when the pull request closes. Reviewers and testers see the change running. Compared with a shared staging environment, previews remove the queue for "who is using staging" and catch integration problems earlier.

To make previews practical:

- Keep them small: use reduced replicas and small databases, or seed data
- Create them from the same manifests as other environments, with parameters
- Give them a time limit and delete them automatically

MARGA by EknathaLabs marga.eknathalabs.com

- Isolate their data, and never copy production data without masking
- Tag every resource with the pull request and owner so cost and cleanup are traceable

Promotion paths

Define how a change moves from development to production: the stages, the checks at each, and what is different. Avoid environment-specific code. Differences should be configuration, not branches. A change should be the same artifact at every stage, so what you tested is what you ship.

Cleanup is a feature

Forgotten environments are one of the largest sources of wasted spend and a security risk, because old environments are rarely patched. Run an automatic job that finds environments past their expiry or without an owner and removes them, with a warning first.

```
from datetime import date, timedelta

today = date(2026, 11, 14)
envs = [
    {"name": "pr-412", "owner": "team-search", "expires": date(2026, 11, 16)},
    {"name": "pr-377", "owner": "team-search", "expires": date(2026, 10, 30)},
    {"name": "demo-old", "owner": None, "expires": None},
]

for e in envs:
    if not e["owner"] or not e["expires"]:
        print(e["name"], "no owner or expiry: warn and schedule removal")
    elif e["expires"] < today:
        print(e["name"], "expired", (today - e["expires"]).days, "days ago: delete")
    elif e["expires"] - today < timedelta(days=3):
        print(e["name"], "expires soon: notify owner")
    else:
        print(e["name"], "ok")
```

When soft isolation is not enough

Namespaces share a kernel, nodes and often a control plane. If tenants are untrusted, handle sensitive data or must meet strict compliance, use stronger isolation: separate node pools, sandboxed runtimes, virtual clusters or separate clusters. Decide this from a threat model, not from convenience.

Diagram: A tenant and its environments

Part	What it does	What goes wrong
Team onboarding	One definition creates the tenant.	Manual setup that differs per team.
Namespaces and quotas	Isolation, limits and default labels.	Missing network policy.
Preview environments	One per pull request, deleted on close.	Never cleaned up.
Shared staging	Integration tests before release.	Everyone fights for it.
Production	Same artifact, different configuration.	Environment-specific code.
Cleanup job	Removes expired or ownerless environments.	Not running, or too gentle.

Flow: Team onboarding to Namespaces and quotas / Namespaces and quotas to Preview environments / Namespaces and quotas to Shared staging / Preview environments to Production / Shared staging to Production / Cleanup job to Preview environments / Cleanup job to Shared staging

Incident: The preview that outlived the pull request

Incident: *A cloud bill jumped by 18 percent over two months. The cause was hundreds of preview environments that nobody deleted.*

```
$ platformctl envs --older-than 14d --count
preview environments older than 14d: 312
estimated monthly cost: 9,400
```

MARGA by EknathaLabs marga.eknathalabs.com

```
owners missing: 41
(this is a simulated illustration, not a real incident)
```

The preview workflow created environments on pull request open but relied on a webhook to delete them on close, and the webhook silently failed for merged branches. The team added a time-to-live label to every preview, a nightly sweep that deleted anything expired, and a weekly report of the count and cost, so the next leak would show up in days.

Common mistakes

- Treating a namespace as a security boundary without quotas, policies and RBAC
- Creating environments by hand with different settings
- Allowing previews with no expiry
- Copying unmasked production data into test environments
- Using code branches instead of configuration to handle environment differences

Check yourself

What turns a namespace into a usable tenant boundary?

Answer: RBAC, quotas, network policies, pod security rules and default labels

What is a preview environment?

Answer: A short-lived copy of the app for one change, deleted when the change closes

Why should differences between environments be configuration, not code?

Answer: So the artifact you tested is the artifact you ship

When is namespace-level isolation not enough?

Answer: For untrusted tenants, sensitive data or strict compliance

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Cost allocation, showback and unit economics

13 minutes. Make cloud and cluster spend visible to the teams that cause it: tagging and labelling, shared cost splitting, showback versus chargeback, and unit costs that connect spend to the product.

Shared platforms hide cost. When twenty teams run on one cluster, the bill arrives as a single number, and nobody feels responsible for it. Cost allocation breaks the bill down to the teams and services that caused it, and gives them data they can act on. It is a core platform capability because only the platform can label everything consistently from the start.

What you will learn

- How to label resources so cost can be attributed
- How to split shared and idle costs fairly
- The difference between showback and chargeback
- How to express cost per unit of product value

Labels first

Allocation depends on consistent labels or tags on cloud resources and on Kubernetes workloads: owner team, service, environment and, where relevant, cost centre. The platform should apply these automatically from the catalog, as part of templates and policies, and reject resources without them. Retrofitting labels is slow and always incomplete.

Cloud providers support cost allocation tags in their billing tools. For Kubernetes, tools such as OpenCost, a CNCF project, estimate cost per namespace, workload or label by combining resource usage with node prices. The FinOps Foundation publishes a framework of practices and terms for this area.

Direct and shared costs

Some costs belong clearly to one service: its database, its storage. Others are shared: the cluster control plane, networking, monitoring, the platform team's own tools. You need a rule for splitting the shared part. Common choices are proportional to usage, such as CPU and memory requested, equal shares, or a fixed percentage. Pick one, publish it, and apply it consistently. A rule people can understand is better than a perfect but obscure one.

Idle and unallocated cost

Reserved capacity that nobody is using still costs money. Track it as its own line: idle node capacity, unattached disks, empty reserved instances. If you hide it by spreading it evenly, no one has a reason to reduce it. Showing it separately lets the platform team own the efficiency of the shared layer.

Showback and chargeback

- **Showback** shows each team the cost it caused, without moving money. It informs and builds awareness.
- **Chargeback** actually bills the team's budget. It creates stronger incentives but also arguments over the splitting rules, so you need accurate data first.

Most organisations start with showback and move to chargeback only when the data is trusted and there is a business reason. Either way, give teams a monthly view they can drill into, with trends and anomalies.

A simple allocation

The snippet below splits a shared cluster bill by requested CPU and shows the unallocated portion.

```
cluster_cost = 12000.0
requested_cpu = {"payments": 120, "search": 80, "checkout": 40}
node_cpu_total = 400  # total capacity, including unused
```

MARGA by EknathaLabs marga.eknathalabs.com

```
allocated = sum(requested_cpu.values())
unit = cluster_cost / node_cpu_total
for team, cpu in requested_cpu.items():
    print(f"{team:10} {cpu * unit:8.2f}")
idle = (node_cpu_total - allocated) * unit
print(f"{'idle':10} {idle:8.2f}")
```

Here costs are split by requested capacity against total capacity, so idle capacity appears as its own line.

Unit economics

Total cost does not say whether spend is healthy. A team whose cost doubled because traffic tripled is doing well. Express cost per unit of value: per thousand requests, per active user, per build, per gigabyte processed. Plot it over time. A rising unit cost is a signal to investigate; a falling one shows efficiency gains. Choose a unit that the product team recognises.

Give actionable advice

Reports that only show numbers are ignored. Pair data with suggestions: right-size requests that are consistently far above usage, remove idle environments, move batch work to cheaper capacity, and use autoscaling. But protect reliability: do not recommend cuts that remove headroom for production traffic.

Keep the platform's own cost visible

The platform also costs money. Report what it costs per supported team or per deploy, and what it saves. This keeps the conversation about value, not only about spend.

Diagram: From bill to team view

Part	What it does	What goes wrong
Cloud bill	Raw cost by account and service.	Single unexplained total.
Labels and tags	Owner, service and environment.	Missing or inconsistent labels.
Usage data	Requests and consumption per workload.	No usage data, only totals.
Allocation rules	Direct, shared and idle costs.	Rules nobody understands.
Team cost view	Showback with trends and unit costs.	Numbers with no advice.
Actions	Right-size, clean up and plan.	Cutting reliability headroom.

Flow: Cloud bill to Allocation rules / Labels and tags to Allocation rules / Usage data to Allocation rules / Allocation rules to Team cost view / Team cost view to Actions

Incident: The unlabelled third

> **Incident:** *Finance asked which team caused a 30 percent rise in monthly spend. The platform team could attribute only two thirds of the bill.*

```
$ costctl report --month 2026-10 --group-by team
payments        41,200
search          28,900
checkout        19,400
unallocated     36,300
(this is a simulated illustration, not a real incident)
```

A large share of resources had been created before labelling was required, and several automation scripts created resources without tags. The team added a policy that denied unlabelled resources, back-filled labels from the catalog using resource names, and reported unallocated spend every week as a number the platform team had to reduce.

Common mistakes

- Starting allocation after resources already exist without labels
- Hiding idle capacity inside team bills
- Using chargeback before the data is trusted

MARGA by EknathaLabs marga.eknathalabs.com

- Reporting only total cost, not cost per unit of value
- Recommending savings that remove production headroom

Check yourself

Why must labels be applied automatically by the platform?

Answer: Retrofitting is slow and always incomplete

What is the difference between showback and chargeback?

Answer: Showback shows teams their cost, chargeback bills their budget

Why show idle capacity as its own line?

Answer: So someone owns the efficiency of the shared layer

Why use unit cost such as cost per thousand requests?

Answer: It shows whether spend is healthy relative to what the product delivers

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Measuring developer experience: DORA, SPACE and cognitive load

14 minutes. Prove the platform helps with a small set of honest measures: delivery metrics from DORA, the dimensions of the SPACE framework, onboarding time and developer surveys, and learn which numbers mislead.

A platform team that cannot show whether it helps will lose its budget to the first reorganisation. A platform team that measures the wrong things will optimise for the wrong outcome. The aim is a few measures that tell you whether developers can deliver changes safely and with less pain, and that you cannot easily game.

What you will learn

- The DORA software delivery measures and how to collect them
- The dimensions of the SPACE framework
- Onboarding time and survey questions that reveal friction
- Common traps when measuring people and teams

DORA delivery measures

The DevOps Research and Assessment programme studied thousands of teams and found a small set of measures that relate to delivery performance:

- **Deployment frequency**: how often a team releases to production
- **Lead time for changes**: how long from a commit to running in production
- **Change failure rate**: the share of releases that cause a failure needing a fix
- **Time to restore**: how quickly service recovers after a failed release or incident

Newer reports refine the names and add related measures, so check the current definitions at dora.dev. The first two describe speed and the last two describe stability. The research finding that matters most is that speed and stability tend to rise together, not trade off, in high performing teams.

You can compute these from data the platform already holds: deployment events from the pipeline, commit times from source control, and incident or rollback records. Measure them per service or team, not as a company-wide average that hides who needs help.

```
from datetime import datetime
from statistics import median

deploys = [
    {"commit": datetime(2026, 11, 2, 9, 0), "live": datetime(2026, 11, 2, 11, 30), "failed": False \
    },
    {"commit": datetime(2026, 11, 3, 14, 0), "live": datetime(2026, 11, 3, 15, 0), "failed": True} \
    ,
    {"commit": datetime(2026, 11, 5, 10, 0), "live": datetime(2026, 11, 5, 16, 0), "failed": False \
    },
    {"commit": datetime(2026, 11, 9, 8, 0), "live": datetime(2026, 11, 9, 8, 45), "failed": False} \
    ,
]
days = 14

lead_hours = [(d["live"] - d["commit"]).total_seconds() / 3600 for d in deploys]
print("deploys per week:", round(len(deploys) / (days / 7), 1))
print("median lead time (hours):", round(median(lead_hours), 1))
print("change failure rate:", sum(d["failed"] for d in deploys) / len(deploys))
```

The SPACE framework

Researchers from Microsoft, GitHub and the University of Victoria proposed SPACE as a way to think about developer productivity beyond one number. It covers five dimensions:

- **Satisfaction and well-being**: how developers feel about their work and tools

MARGA by EknathaLabs
marga.eknathalabs.com

- **Performance**: outcomes such as quality and impact
- **Activity**: counts of actions such as commits or deploys
- **Communication and collaboration**: how well people and teams work together
- **Efficiency and flow**: how smoothly work proceeds with few interruptions and waits

The core advice is to measure across several dimensions, and to include at least one perception measure. Activity counts alone, such as lines of code or number of commits, are easy to collect and easy to game.

Cognitive load and onboarding

Two measures speak directly to what a platform does. **Time to first deploy** for a new engineer, from laptop setup to a change running in a real environment, shows how much the platform removes. **Time to create a new service** shows the cost of starting something. Track both as medians, and run a short survey after each task asking how hard it was and what slowed the person down.

A useful survey is brief and recurring: how easy is it to deploy, to debug in production, to find documentation, to get help? Add a free-text question. A rating alone tells you that something is wrong; the comment tells you what.

Treat adoption as evidence, not proof

Rising usage of the platform is good news but not proof of value, especially if it is mandatory. Pair it with satisfaction and outcome measures. If usage is high and satisfaction is low, people are tolerating it.

Traps to avoid

- **Targets become goals.** When a measure becomes a target, people optimise the number. Use measures to learn and discuss, not to rank individuals.
- **Comparing teams unfairly.** A team maintaining a regulated mainframe interface has different constraints from a new web service.
- **Averages that hide pain.** Use medians and percentiles, and look at the slowest cases.
- **Measuring only what is easy.** Deploy counts are easy; the feeling of being blocked is not, and it often matters more.
- **Surveillance.** Individual tracking harms trust. Measure systems and teams.

Close the loop

Review the measures with the product teams every quarter. Pick one or two to improve, make a hypothesis about what platform change will move them, and check afterwards. The conversation is the product of measurement, not the dashboard.

Diagram: Measuring whether the platform helps

Part	What it does	What goes wrong
Platform data	Deploy events, commits and incidents.	Data stored per tool and never joined.
Delivery measures	Frequency, lead time, failure rate and recovery.	Company-wide averages hiding outliers.
Onboarding time	Time to first deploy and to a new service.	Measured once, never repeated.
Developer survey	Ease, blockers and free text.	Rare surveys with no follow-up.
Quarterly review	Pick one or two things to improve.	Dashboards without decisions.
Platform change	A hypothesis tested against the measures.	Changes made without a baseline.

Flow: Platform data to Delivery measures / Platform data to Onboarding time / Delivery measures to Quarterly review / Onboarding time to Quarterly review / Developer survey to Quarterly review / Quarterly review to Platform change

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The leaderboard

> **Incident:** *A manager published a ranking of teams by deployment count. Within a month, teams were splitting changes into tiny deploys, and change failure rate did not improve.*

```
$ platformctl metrics --team search --last 30d
deploys:            94   (previous 30d: 41)
median lead time:   2.9h (previous: 3.1h)
change failure:     17%  (previous: 16%)
rollbacks:          15   (previous: 7)
(this is a simulated illustration, not a real incident)
```

Deployment frequency doubled but the stability numbers were unchanged or worse, which is a sign of gaming. The platform team withdrew the leaderboard, replaced it with per-team trends that only the team and its manager saw, and added the survey question "what slows you down most?" to the quarterly review.

Common mistakes

- Ranking individuals or teams by a single activity count
- Using a company-wide average that hides problem teams
- Ignoring the human side: surveys and free-text feedback
- Treating mandatory usage as success
- Changing the platform with no baseline to compare against

Check yourself

Which pair of DORA measures describe stability?

Answer: Change failure rate and time to restore

Why does SPACE advise measuring several dimensions?

Answer: One number is easy to game and misses how work actually feels and flows

Which measure speaks most directly to what a platform removes?

Answer: Time to first deploy for a new engineer

Why avoid ranking teams by deployment count?

Answer: People game the number without improving outcomes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Platform SLOs, support and on-call for the platform team

13 minutes. The platform is production software with internal customers. Set service level objectives for it, run a support model that scales, and decide what the platform team is on call for and what product teams own.

When the pipeline is down, every team stops shipping. When the cluster API is slow, every deploy fails. The platform is a dependency of everything else, which makes its reliability matter more than that of any single product. Treat it like a service with customers, objectives, an error budget and an on-call rotation.

What you will learn

- Which platform capabilities need SLOs, and what to measure
- How to design support so it does not drown the team
- What the platform team owns versus what product teams own
- How to communicate incidents and changes

SLOs for platform capabilities

Pick the capabilities users depend on most and describe their reliability in terms users feel:

Capability	Example indicator	Example objective
Pipeline	Share of builds that start within 2 minutes	99 percent over 30 days
Deploy	Share of deploys that succeed without platform error	99.5 percent
Cluster API	Share of requests under 1 second	99.9 percent
Self-service workflows	Time from request to ready for common tasks	95 percent under 15 minutes
Portal	Availability of the catalog and templates	99.5 percent

Choose indicators that exclude user errors, such as a build failing because of a user's code, and include platform faults. Set objectives with the users, based on how much downtime they can tolerate. Do not aim for the highest number you can imagine, because each extra nine costs a great deal.

Error budgets for the platform

An error budget is the amount of unreliability the objective allows. If the pipeline objective is 99 percent of builds starting within two minutes over thirty days, the budget is one percent. When the budget is nearly spent, the platform team slows feature work and fixes reliability. When the budget is healthy, it can ship changes faster. This gives platform decisions the same discipline that the SRE practice gives to products.

```
window_minutes = 30 * 24 * 60
objective = 0.995
budget_minutes = window_minutes * (1 - objective)
bad_minutes = 85

remaining = budget_minutes - bad_minutes
print("budget (min):", round(budget_minutes))
print("used (min):", bad_minutes)
print("remaining (min):", round(remaining))
print("status:", "slow down changes" if remaining < budget_minutes * 0.25 else "ok to ship")
```

A support model that scales

Without a plan, support consumes the whole team. A workable structure has layers:

1. **Self-service documentation** that answers the common questions, kept current from the questions actually asked

2. **A request channel or form** with templates, so each question arrives with the information needed
3. **A rotating support engineer** who handles incoming questions for a week, so the rest of the team can focus
4. **Escalation to the owning engineers** for real defects

Categorise questions. If the same question appears ten times, fix the cause: improve the template, error message or documentation. The goal is a falling number of questions per user.

Who is on call for what

Write a clear boundary. The platform team is on call for the platform's own components: the pipeline system, the cluster control plane, the portal, shared databases, and the workflows. Product teams are on call for their services, including those that run on the platform. A service failing because of its own code is not a platform incident, even though it runs on the platform's cluster. Make this boundary visible, with example scenarios, so nobody wastes the first twenty minutes arguing about whose problem it is.

Give product teams good tools to diagnose their own services, such as default dashboards and logs, so they can tell whether the problem is theirs or the platform's.

Communicate well

Maintain a status page for the platform. During incidents, post updates at a regular cadence. Announce planned maintenance and upgrades in advance, with a date, the expected impact and what users must do. After a significant incident, publish a short blameless review. People forgive outages they understand and hate surprises.

Plan upgrades as a service

The platform depends on clusters, controllers and tools that need upgrades. Give each a schedule, test upgrades in a non-production copy, and use staged rollouts. Tell teams what changes for them, and provide automated fixes where possible.

Diagram: Running the platform as a service

Part	What it does	What goes wrong
SLOs	Targets for pipeline, deploy and workflows.	Targets set without users.
Error budget	Decides pace of change versus reliability.	Ignored when spent.
Support model	Docs, forms, rotation and escalation.	One person answers everything.
Platform on call	Owns platform components.	Unclear boundary with product teams.
Status and notes	Status page, notices and reviews.	Silence during incidents.

Flow: SLOs to Error budget / Error budget to Support model / Error budget to Platform on call / Support model to Status and notes / Platform on call to Status and notes

Incident: Whose problem is it?

> **Incident:** *At 11:40 several teams reported failing deploys. The platform on-call spent half an hour on a pipeline that was healthy, while a different cause was fixing itself.*

```
$ platformctl status pipeline
build queue:      healthy (median wait 41s)
deploy workers:   healthy
registry:         degraded: rate limit from upstream
(this is a simulated illustration, not a real incident)
```

The failure was a rate limit on pulls from a public registry, which showed up as deploy failures. The team added the registry to the platform's own indicators, put a pull-through cache in front of it, and updated the status page template to show dependencies on outside services.

Common mistakes

- Setting an SLO nobody asked for and nobody measures
- Ignoring the error budget when it is spent
- Letting every question go straight to senior engineers
- Leaving the on-call boundary between platform and products vague
- Going silent during platform incidents

Check yourself

What should an indicator for a platform SLO exclude?

Answer: Failures caused by the user's own code

What happens when the error budget is nearly spent?

Answer: The team slows feature work and invests in reliability

How should support scale?

Answer: Self-service docs, request forms, a rotating support engineer and escalation

Who is on call for a product service running on the platform?

Answer: The product team that owns the service

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Adoption, migration and deprecation

13 minutes. A platform only pays off when teams use it and old ways are retired. Plan adoption like a rollout, migrate services in waves with tooling and support, and deprecate old paths with dates, not hopes.

Building a platform is half of the work. The other half is getting teams from where they are to where the platform is, and then switching off the old ways. Companies that skip this end up paying for both worlds: the new platform and the old pipelines, scripts and servers that nobody dared remove.

What you will learn

- How to plan adoption with pilots and waves
- What a migration kit contains
- How to deprecate responsibly
- How to deal with teams that cannot or will not move

Start with a pilot

Pick two or three teams that have a real need, a friendly attitude and a service that matters but is not the most critical. Work with them closely, fix what breaks, and record how long each step took. Their results become the story and the checklist for everyone else. Avoid starting with the most critical system, where any bug is expensive, or with the least engaged team, which teaches you little.

Migrate in waves

After the pilot, group the remaining services by effort and risk. Early waves take services that match the golden path well. Later waves take the awkward ones: unusual runtimes, stateful systems, regulated workloads. For each wave, schedule dates, name a migration contact on each team and assign a platform engineer who can help. A wave plan with dates turns "we should move" into a commitment.

The migration kit

Make moving cheap. A kit usually includes:

- An inventory of what each service uses today, found by scanning repositories and pipelines
- A step-by-step guide with an estimate of effort
- Automation that opens a pull request adding the new pipeline and catalog descriptor
- A checklist of verification steps to compare old and new behaviour
- A rollback plan so teams are not afraid to try

Offer office hours or embedded help for the first weeks. The point is to reduce the effort a team needs to put in.

Track adoption honestly

Track the share of services on the platform, by tier and by team, as well as how recently they were updated. Also track the reverse: who moved and then left again, and why. The snippet below summarises a migration list by wave.

```
services = [
    {"name": "checkout", "wave": 1, "status": "done"},
    {"name": "search", "wave": 1, "status": "done"},
    {"name": "billing", "wave": 2, "status": "in progress"},
    {"name": "reports", "wave": 2, "status": "blocked"},
    {"name": "mainframe-bridge", "wave": 3, "status": "not started"},
]

waves = {}
for s in services:
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    w = waves.setdefault(s["wave"], {"total": 0, "done": 0})
    w["total"] += 1
    w["done"] += s["status"] == "done"
for wave, w in sorted(waves.items()):
    print(f"wave {wave}: {w['done']}/{w['total']} done")
print("blocked:", [s["name"] for s in services if s["status"] == "blocked"])
```

Deprecate with dates

Deprecation is a promise with a schedule. Announce what will be retired, why, when and what to do instead. A typical sequence is:

1. Announce with the date and the migration guide
2. Stop creating new things on the old path
3. Warn when the old path is used, with a link to the guide
4. Remove support and switch the old path off on the announced date

Stick to the date, but build in a clear process for exceptions with owners and expiry. If dates always slip, nobody believes the next one.

Handle the teams that stay

Some teams have good reasons to stay: a system near end of life, a regulatory constraint or an unusual need the platform does not meet. Record the reason. If several teams share it, the platform has a gap worth closing. If a team simply does not want to move, escalate with data on risk, cost and support burden, and agree an end date with its leadership. Do not let a "temporary" exception become permanent.

Keep the old and the new apart

During migration, avoid a half-state where a service is partly on the old path and partly on the new. Define a clear cutover: traffic shifts, the old pipeline is disabled and the old resources are cleaned up. Leave nothing running "just in case" without an owner and a removal date.

Measure the end state

A migration is done when the old system is gone, not when the new one is available. Track the decommission: old pipelines deleted, old servers turned off, old credentials revoked and old documentation archived. Each retired system reduces cost, risk and confusion.

Diagram: Adoption in waves

Part	What it does	What goes wrong
Pilot teams	Two or three willing teams.	Starting with the hardest system.
Migration kit	Inventory, guide, automation and checks.	Manual steps nobody wrote down.
Wave one	Services that fit the path well.	No dates or contacts.
Later waves	Awkward and stateful services.	Postponed forever.
Deprecation	Dates, warnings and removal.	Dates that always slip.
Old system removed	Pipelines, servers and credentials gone.	Left running just in case.

Flow: Pilot teams to Migration kit / Migration kit to Wave one / Migration kit to Later waves / Wave one to Deprecation / Later waves to Deprecation / Deprecation to Old system removed

Incident: The old pipeline that was still deploying

Incident: *Six months after the new platform launched, a security fix reached only part of the fleet, because ten services still deployed through a pipeline that nobody maintained.*

```
$ platformctl audit pipelines --legacy
legacy pipelines still active: 10
last owner change: 14 months ago
```

MARGA by EknathaLabs marga.eknathalabs.com

```
credentials rotated: never
(this is a simulated illustration, not a real incident)
```

The old pipeline had never been listed as something to retire. The platform team inventoried it, announced a removal date, opened pull requests moving the ten services and revoked the old credentials on the date. They added "decommission" as a tracked item to every future migration.

Common mistakes

- Starting with the most critical or the least willing team
- Offering the platform without migration tooling
- Announcing deprecation with no date
- Allowing temporary exceptions to become permanent
- Declaring victory before the old system is removed

Check yourself

What makes a good pilot team?

Answer: A team with a real need and a friendly attitude, with a service that matters but is not the most critical

What should a migration kit include?

Answer: Inventory, guide, automation, verification checks and rollback plan

Why give deprecations a date?

Answer: A promise with a schedule gets planned for, while hopes do not

When is a migration complete?

Answer: When the old system is removed, not when the new one is available

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. Capstone: design and ship a golden path

20 minutes. Bring the whole course together: define a golden path for a web service, build a scaffolder that validates a request and produces a catalog entry and policy-checked manifest, and plan its rollout and measures.

This lesson is a design and build exercise. You will specify one golden path end to end, write a small scaffolder that validates a request and generates the files, check the result against policy, and plan how you would roll the path out and know it works. The code is a toy, but each part is a miniature of something a real platform does.

What you will learn

- How to turn a path specification into automation
- How to generate a catalog entry and a manifest from one request
- How to check the output against policy before it is created
- How to plan adoption and measures for the result

Step 1: Write the path spec

Write one page that answers these questions for the path "web service":

- **Who is it for?** Product teams building stateless HTTP services
- **What do they get?** Repository, pipeline, deployment manifest, catalog entry, dashboard and alerts
- **What is fixed?** Base image, security checks, labels, resource limits
- **What can they change?** Name, team, tier, replicas within a range, extra environment variables
- **What does support look like on and off the path?**
- **What does success look like?** Time to first deploy under an hour and 90 percent of new services created through the path within two quarters

Step 2: Build the scaffolder

The script below validates a request, produces a catalog entry and a deployment manifest as Python structures, and runs the policy checks from earlier lessons. Real systems would write files and call APIs. Here you can read the logic in a few lines.

```
import json
import re

TEAMS = {"payments", "search", "platform"}
TIERS = {"critical": (3, 10), "standard": (2, 6), "experimental": (1, 2)}

def validate(req):
    errors = []
    if not re.fullmatch(r"[a-z][a-z0-9-]{2,29}", req.get("name", "")):
        errors.append("name must be 3-30 lowercase letters, digits or hyphens")
    if req.get("team") not in TEAMS:
        errors.append("unknown team")
    tier = req.get("tier")
    if tier not in TIERS:
        errors.append("tier must be one of " + ", ".join(sorted(TIERS)))
    else:
        lo, hi = TIERS[tier]
        if not lo <= req.get("replicas", lo) <= hi:
            errors.append(f"replicas for {tier} must be {lo}-{hi}")
    return errors

def render(req):
    catalog = {
        "kind": "Component", "name": req["name"], "owner": "team-" + req["team"],
        "lifecycle": "production" if req["tier"] != "experimental" else "experimental",
```

MARGA by EknathaLabs marga.eknathalabs.com

```
        "tier": req["tier"], "links": ["dashboard/" + req["name"], "runbook/" + req["name"]],
    }
    deploy = {
        "name": req["name"], "replicas": req.get("replicas", TIERS[req["tier"]][0]),
        "labels": {"owner": "team-" + req["team"], "tier": req["tier"]},
        "containers": [{"name": "app", "image": "registry.example/" + req["name"] + ":latest",
                        "resources": {"limits": {"memory": "512Mi", "cpu": "500m"}}, "run_as_root \
    ": False}],
    }
    return catalog, deploy

def policy(deploy):
    out = []
    for c in deploy["containers"]:
        if c["image"].endswith(":latest"):
            out.append("image tag latest is not allowed: pin a version")
        if "limits" not in c["resources"]:
            out.append("missing resource limits")
        if c["run_as_root"]:
            out.append("must not run as root")
    if "owner" not in deploy["labels"]:
        out.append("missing owner label")
    return out

req = {"name": "billing", "team": "payments", "tier": "standard", "replicas": 3}
errs = validate(req)
if errs:
    print("REJECTED:", errs)
else:
    catalog, deploy = render(req)
    problems = policy(deploy)
    print(json.dumps({"catalog": catalog["name"], "owner": catalog["owner"], "policy_problems": pr \
    oblems}, indent=2))
```

Run it. The generated manifest uses a `latest` tag, so the policy check reports a problem. This is deliberate: the template has a bug, and the policy check is what finds it before a service is created. Fix the template to use a pinned version, rerun and confirm the problem list is empty.

Step 3: Extend it

Try these extensions, one at a time:

1. Reject a request whose replica count falls outside the tier's range, and test it.
2. Add a check that rejects names already in an existing-services list.
3. Add a `deprecation` field to the catalog entry that the path can set when a path version is retired.
4. Add a function that takes a list of existing services and reports which are missing owner, dashboard or runbook entries.
5. Write three tests: a good request, a bad name and an out-of-range replica count.

Step 4: Plan the rollout

Write down a short plan:

- **Pilot:** which two teams, with what success criteria
- **Waves:** how you will group the remaining services
- **Migration kit:** what you will offer so moving is cheap
- **Measures:** time to first deploy, share of new services on the path, change failure rate and a developer satisfaction question
- **Support:** who answers questions during the first month
- **Deprecation:** what old route you will retire and by when

MARGA by EknathaLabs marga.eknathalabs.com

What to check

A good capstone has a one-page spec, a scaffolder that refuses bad input with clear messages, a policy check that catches an unsafe default, tests, and a rollout plan with dates and measures. Ask a colleague to try your path as a new user and time how long it takes them.

Diagram: The capstone end to end

Part	What it does	What goes wrong
Path specification	Who, what, fixed and flexible, success.	A spec nobody reads.
Request	Name, team, tier and replicas.	Too many questions.
Validate	Naming, team, tier and range checks.	Checks after creation.
Generate	Catalog entry and manifest.	Template with unsafe defaults.
Policy check	Image tag, limits, root and owner.	Policy not run in the template.
Rollout plan	Pilot, waves, kit and measures.	No dates or owners.

Flow: Path specification to Request / Request to Validate / Validate to Generate / Generate to Policy check / Path specification to Rollout plan / Policy check to Rollout plan

Incident: The template that shipped latest

> **Incident:** *A golden path template used an unpinned image tag. For a month, new services deployed whatever the newest image happened to be, and one release broke three services at once.*

```
$ platformctl lint template web-service
FAIL image tag is latest in deploy/manifest.yaml
FAIL no policy check in template pipeline
(this is a simulated illustration, not a real incident)
```

The template had never been tested in CI, and its pipeline did not run the same policy as the cluster. The team added a CI job that generated a sample service and ran the full policy check, pinned the image tag in the template, and added an automatic pull request to update existing services.

Common mistakes

- Writing the template before the spec
- Skipping tests of the template itself
- Running policy only in the cluster, not at creation time
- Giving no plan for existing services
- Measuring only how many services use the template, not whether they are better off

Check yourself

Why write the path specification before the code?

Answer: It states who the path is for, what is fixed and what success looks like

What should the scaffolder do with an invalid request?

Answer: Reject it early with clear messages and create nothing

Why run a policy check at generation time as well as in the cluster?

Answer: Developers see problems in their own change, before deployment

How do you know the golden path works?

Answer: Time to first deploy, share of new services on it, stability measures and developer feedback

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Platform engineering

Platform as a product, golden path checklist, catalog descriptor, self-service design, guardrails, cost labels and measures.

Roles compared

Role	Main question	Typical output
DevOps	How do build and run work together?	Shared practice and culture
SRE	How reliable is the service?	SLOs, error budgets, incident process
Platform	How can teams ship safely with less effort?	A self-service internal product

Golden path checklist

Item	Check
Demand	Many teams do this job repeatedly
Defaults	Secure, observable and cost-labelled out of the box
Escape	A documented way to customise
Updates	Pushed as pull requests, versioned
Evidence	Time to first deploy measured

Catalog descriptor fields

Field	Why it matters
owner	Who gets the page and the questions
lifecycle	Experimental, production or deprecated
tier	How critical the service is
dependsOn	What breaks if it fails
links	Runbook, dashboard, repository

Self-service design rules

Rule	Reason
Ask few questions	Fewer wrong answers
Validate before creating	Failures happen early and cleanly
Make steps idempotent	A retry cannot duplicate or corrupt
Report failure to the requester	Nobody waits in silence
Test the template in CI	A broken template breaks every new service

Policy rollout

Stage	Behaviour
Audit	Record violations only
Warn	Show a message with the fix
Enforce	Reject, with a clear message and a time-limited exception route

MARGA by EknathaLabs marga.eknathalabs.com

Cost labels

Label	Use
owner	Which team pays
service	Which product it supports
environment	Production, staging or preview
expires	When a preview should be removed

Measures

Measure	Reads as
Deployment frequency	How often value ships
Lead time for changes	How fast a change reaches production
Change failure rate	How often a change causes harm
Time to restore	How fast service recovers
Time to first deploy	How easy the platform is for a new service

MARGA by EknathaLabs
marga.eknathalabs.com